MLOPS Assignment 2

Name: Spoorthi K.S

USN: 4GW23CI052

1. Which run had the lowest latency, and why might that be?

In the latest execution, **run_batch4** had the lowest latency (0.325 s), followed by run_batch8 (0.520 s) and run_batch1 (1.299 s). Batching lets the model process several sentences in one forward pass, so there are fewer inference calls and less per-call overhead (tokenisation, Python and framework overhead). run_batch1 makes 10 separate calls for 10 sentences, so it is the slowest. run_batch4 may also have edged out run_batch8 because it ran last, after the model and caches were already warm, and because 10 samples split into batches of 4 (4+4+2) avoid the padding cost of a larger batch. The exact timings vary a little from execution to execution (in the first execution of this session, run_batch1 took 1.48 s and run_batch8 took 0.621 s), but the trend is the same: larger batches are faster.

2. Did accuracy change between runs? Why or why not?

No. Accuracy stayed at **1.0 (100%)** in every run. All runs used the same pre-trained model (distilbert-base-uncased-finetuned-sst-2-english) with the same weights on the same 10 labelled sentences, and inference is deterministic. Batch size only changes how the inputs are grouped and how fast they are processed, not the model or its predictions.

3. Name two benefits of using MLflow here instead of printing results in the terminal.

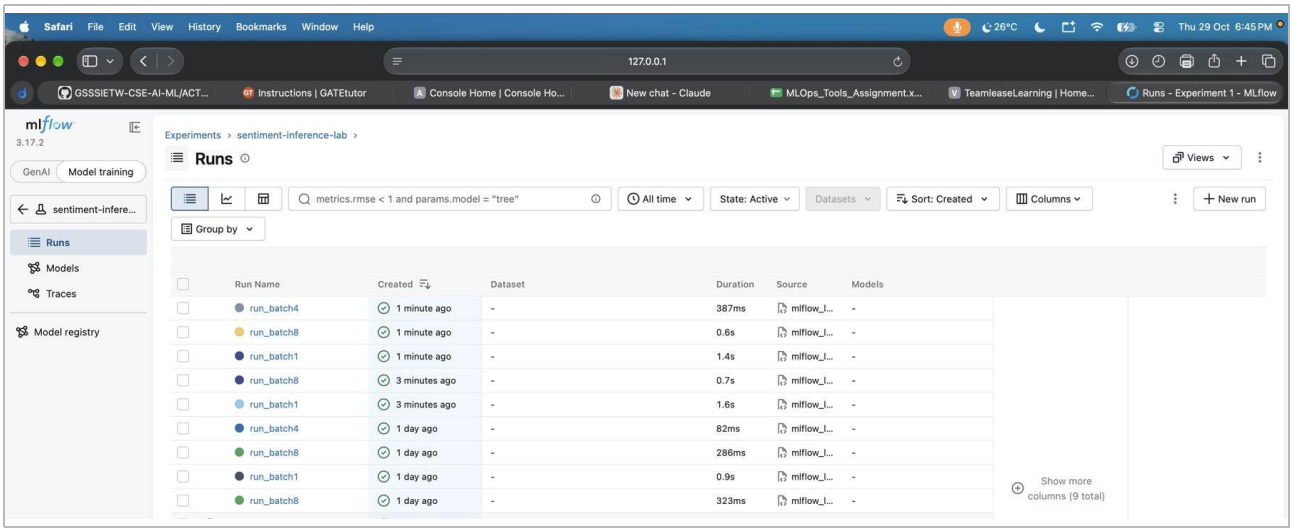
• **Reproducibility and record-keeping:** MLflow stores the parameters (model_name, batch_size, num_samples), metrics (accuracy, avg_confidence, total_latency_sec) and artifacts (predictions.csv) of every run permanently. Terminal output is lost once the window is cleared or closed.

• **Easy comparison:** runs can be compared side by side in the UI with tables and charts (parallel coordinates, scatter plots), which is much easier than reading printed lines.

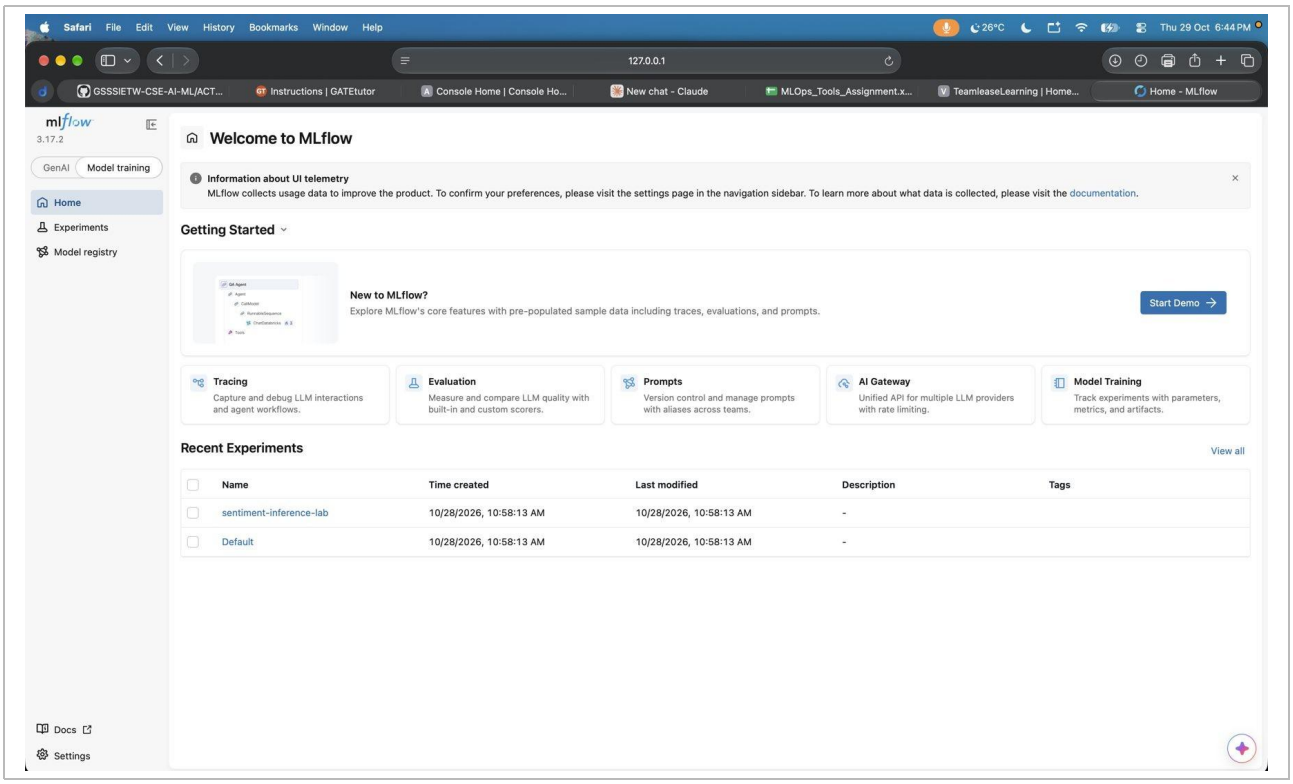
Results summary (latest execution)

Run	batch_size	Accuracy	Latency (s)
run_batch1	1	1.0	1.299
run_batch8	8	1.0	0.520
run_batch4	4	1.0	0.325

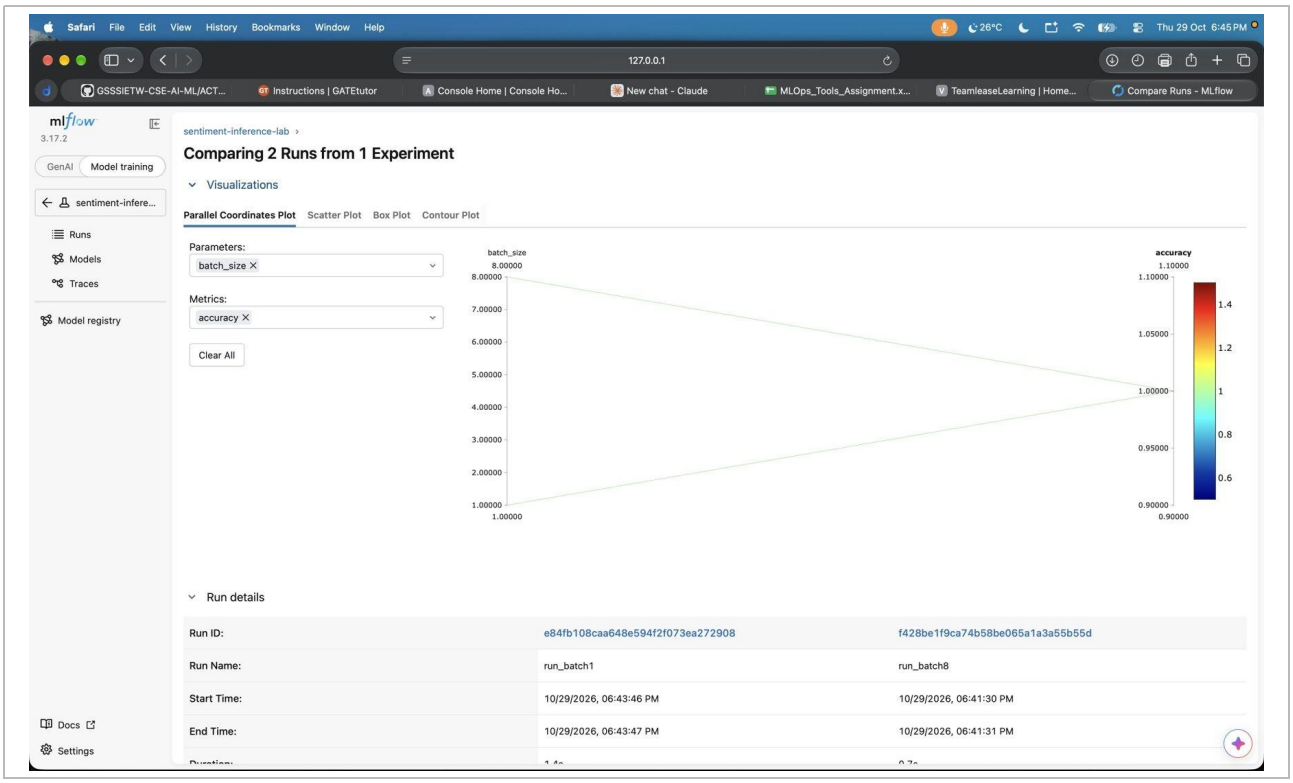
Screenshot of the MLflow runs table:



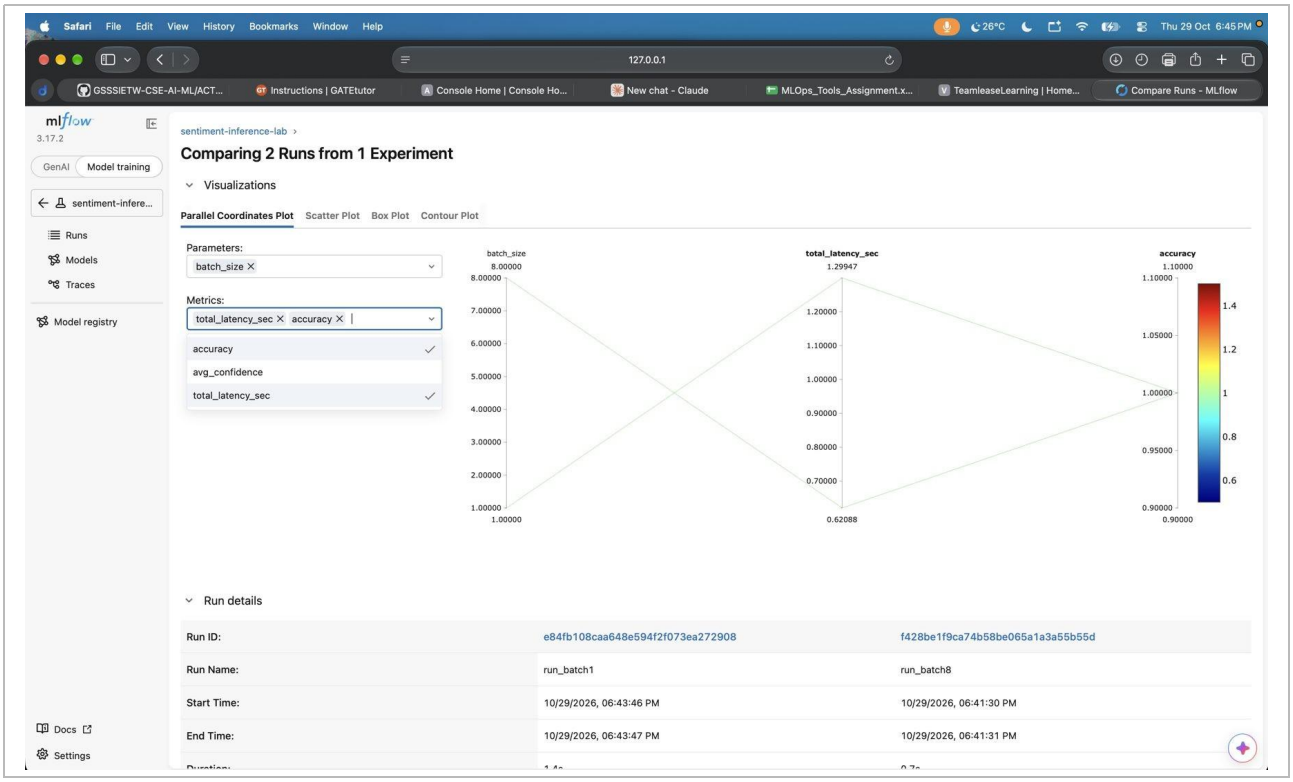
Screenshot of the MLflow home page (experiment sentiment-inference-lab):



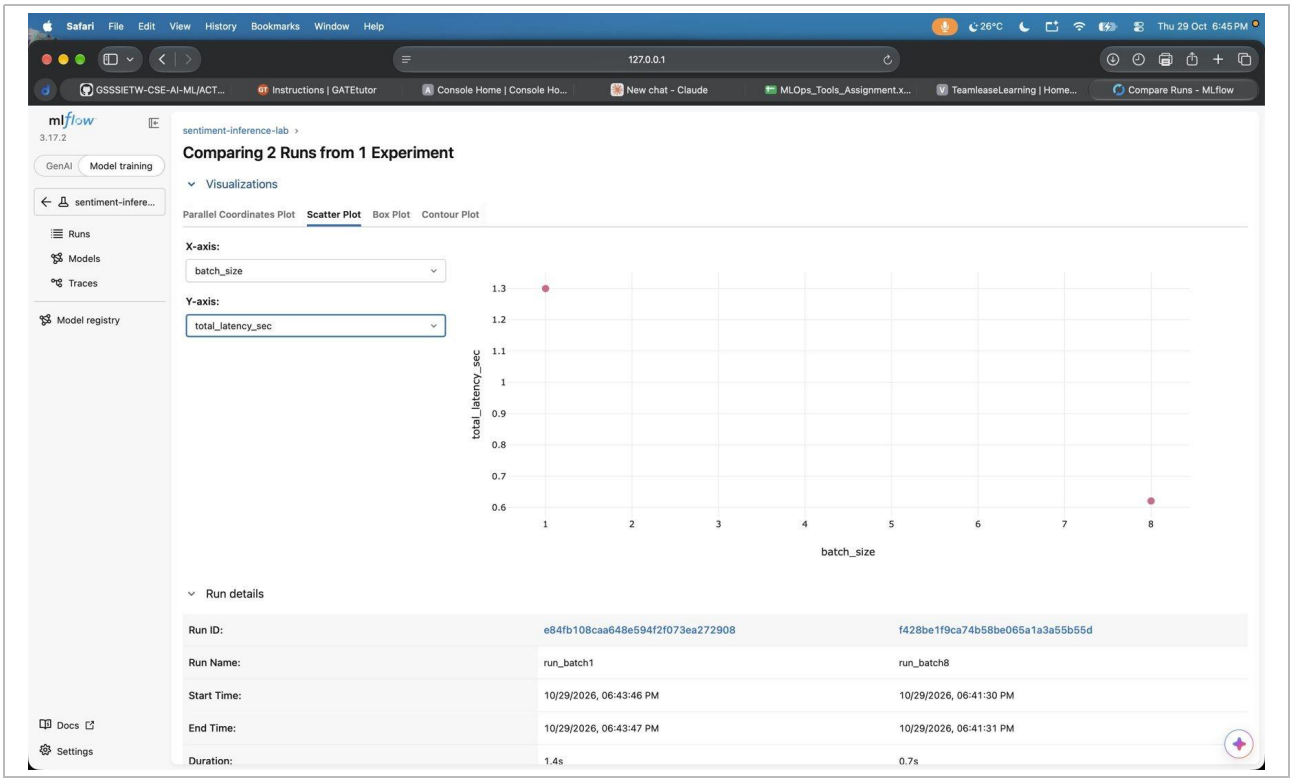
Screenshot of the Compare view (batch_size vs accuracy):



Screenshot of the Compare view (batch_size vs total_latency_sec and accuracy):



Screenshot of the scatter plot (batch_size vs total_latency_sec):



mlflow_lab.py file:

```
import time
import mlflow
import pandas as pd
from transformers import pipeline

# Small labelled dataset (10 sentences)
data = [
    ("I loved this movie, it was fantastic!", "POSITIVE"),
    ("The service was terrible and slow.", "NEGATIVE"),
    ("What a wonderful experience.", "POSITIVE"),
    ("I regret buying this product.", "NEGATIVE"),
    ("The food was delicious and fresh.", "POSITIVE"),
    ("This is the worst app I have ever used.", "NEGATIVE"),
    ("Absolutely brilliant performance by the team.", "POSITIVE"),
    ("The battery dies within an hour.", "NEGATIVE"),
    ("Great value for money, highly recommend.", "POSITIVE"),
    ("I am very disappointed with the quality.", "NEGATIVE"),
]

texts = [text for text, _ in data]
labels = [label for _, label in data]

mlflow.set_experiment("sentiment-inference-lab")

def run_experiment(model_name, batch_size, run_name):
    clf = pipeline("sentiment-analysis", model=model_name)

    with mlflow.start_run(run_name=run_name):
        # Parameters
        mlflow.log_param("model_name", model_name)
        mlflow.log_param("batch_size", batch_size)
        mlflow.log_param("num_samples", len(texts))

        # Inference
        start = time.time()
        preds = clf(texts, batch_size=batch_size)
        latency = time.time() - start

        pred_labels = [prediction["label"] for prediction in preds]
        accuracy = sum(predicted == actual for predicted, actual in zip(pred_labels, labels)) / len(labels)
        avg_confidence = sum(prediction["score"] for prediction in preds) / len(preds)

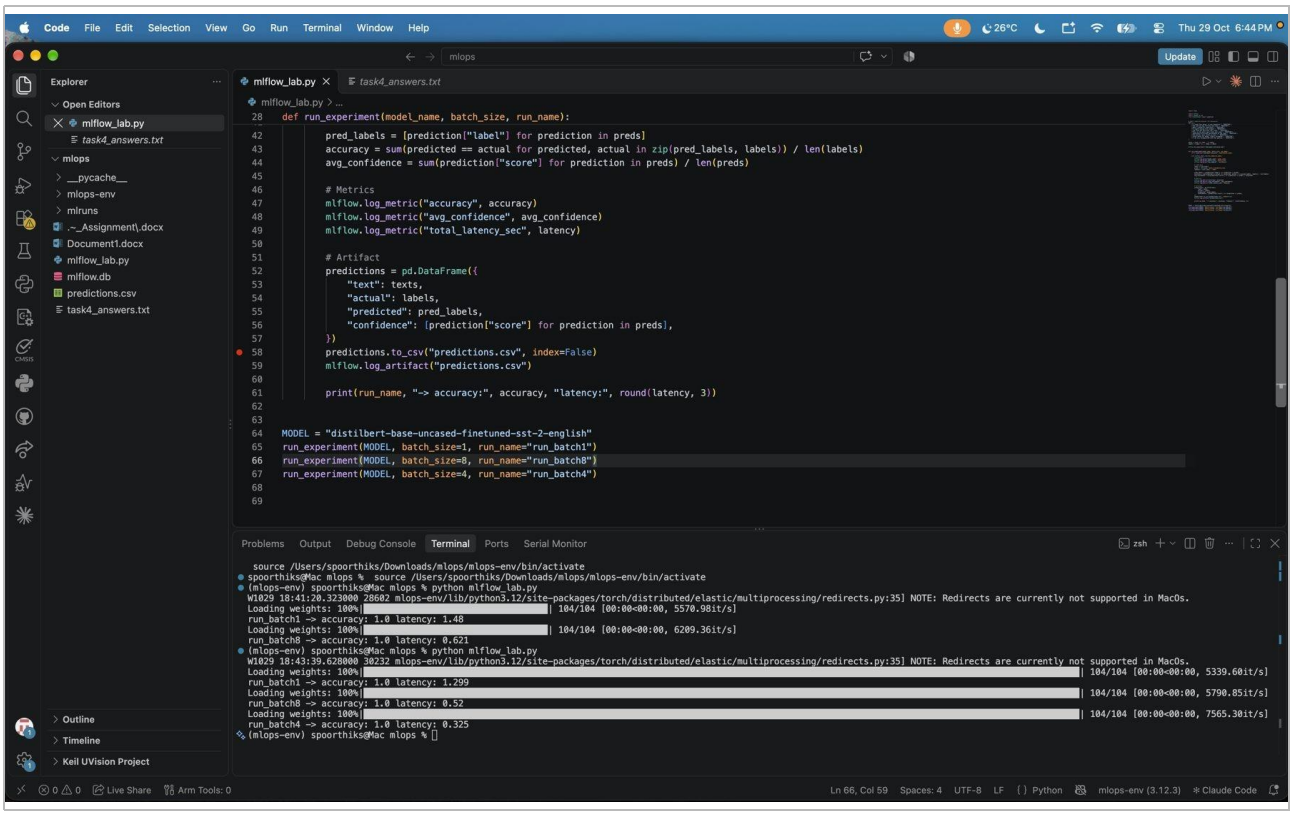
        # Metrics
        mlflow.log_metric("accuracy", accuracy)
        mlflow.log_metric("avg_confidence", avg_confidence)
        mlflow.log_metric("total_latency_sec", latency)

        # Artifact
        predictions =
            pd.DataFrame({ "text":
            texts,
            "actual": labels,
            "predicted": pred_labels,
            "confidence": [prediction["score"] for prediction in preds],
        })
        predictions.to_csv("predictions.csv", index=False)
        mlflow.log_artifact("predictions.csv")

        print(run_name, "-> accuracy:", accuracy, "latency:", round(latency, 3))

MODEL = "distilbert-base-uncased-finetuned-sst-2-english"
run_experiment(MODEL, batch_size=1, run_name="run_batch1")
run_experiment(MODEL, batch_size=8, run_name="run_batch8")
run_experiment(MODEL, batch_size=4, run_name="run_batch4")
```

Execution:


Code File Edit Selection View Go Run Terminal Window Help
Thu 29 Oct 6:44 PM
mlops
Update
Explorer
Open Editors
mlflow_lab.py
task4_answers.txt
mlops
__pycache__
mlops-env
mlruns
.~_Assignment\.docx
Document1.docx
mlflow_lab.py
mlflow.db
predictions.csv
task4_answers.txt
Outline
Timeline
Keil UVision Project
mlflow_lab.py
task4_answers.txt
mlflow_lab.py > ...
def run_experiment(model_name, batch_size, run_name):
pred_labels = [prediction["label"] for prediction in preds]
accuracy = sum(predicted == actual for predicted, actual in zip(pred_labels, labels)) / len(labels)
avg_confidence = sum(prediction["score"] for prediction in preds) / len(preds)
Metrics
mlflow.log_metric("accuracy", accuracy)
mlflow.log_metric("avg_confidence", avg_confidence)
mlflow.log_metric("total_latency_sec", latency)
Artifact
predictions = pd.DataFrame({
"text": texts,
"actual": labels,
"predicted": pred_labels,
"confidence": [prediction["score"] for prediction in preds],
})
predictions.to_csv("predictions.csv", index=False)
mlflow.log_artifact("predictions.csv")
print(run_name, "-> accuracy:", accuracy, "latency:", round(latency, 3))
MODEL = "distilbert-base-uncased-finetuned-sst-2-english"
run_experiment(MODEL, batch_size=1, run_name="run_batch1")
run_experiment(MODEL, batch_size=8, run_name="run_batch8")
run_experiment(MODEL, batch_size=4, run_name="run_batch4")
Problems Output Debug Console Terminal Ports Serial Monitor
source /Users/spoorthiks/Downloads/mlops/mlops-env/bin/activate
spoorthiks@Mac mlops % source /Users/spoorthiks/Downloads/mlops/mlops-env/bin/activate
(mlops-env) spoorthiks@Mac mlops % python mlflow_lab.py
W1029 18:41:20.323000 28602 mlops-env/lib/python3.12/site-packages/torch/distributed/elastic/multiprocessing/redirects.py:35] NOTE: Redirects are currently not supported in MacOs.
Loading weights: 100%| | 104/104 [00:00<00:00, 5570.98it/s]
run_batch1 -> accuracy: 1.0 latency: 1.48
Loading weights: 100%| | 104/104 [00:00<00:00, 6209.36it/s]
run_batch8 -> accuracy: 1.0 latency: 0.621
(mlops-env) spoorthiks@Mac mlops % python mlflow_lab.py
W1029 18:43:39.628000 30232 mlops-env/lib/python3.12/site-packages/torch/distributed/elastic/multiprocessing/redirects.py:35] NOTE: Redirects are currently not supported in MacOs.
Loading weights: 100%| | 104/104 [00:00<00:00, 5339.60it/s]
run_batch1 -> accuracy: 1.0 latency: 1.299
Loading weights: 100%| | 104/104 [00:00<00:00, 5790.85it/s]
run_batch8 -> accuracy: 1.0 latency: 0.52
Loading weights: 100%| | 104/104 [00:00<00:00, 7565.30it/s]
run_batch4 -> accuracy: 1.0 latency: 0.325
(mlops-env) spoorthiks@Mac mlops %
Live Share
Arm Tools: 0
Ln 66, Col 59
Spaces: 4
UTF-8
LF
Python
mlops-env (3.12.3)
Claude Code